Projet d'optimisation numérique

Rapport Python

Bocoum Amadou

Anya Levêque

April 19, 2026

Sommaire

1 Projet d'optimisation numérique **2**

1.1 Rapport Python 2

1.1.1 Auteurs 2

1.1.2 Rappel mathématique : méthodologie de l'annexe 3

1.2 Exercice 1 3

1.3 Exercice 2 6

1.4 Exercices 3 à 5 7

1.4.1 Guide de lecture des exercices 2 à 5 7

2 Exercice 2: **10**

3 Exercice 3: **11**

4 Exercice 4: **11**

5 Exercice 5: **11**

6 Synthèse: **11**

6.1 Exercices 6 et 7 12

6.1.1 Modèle statistique et séparation des exercices 6 et 7 12

6.2 Exercice 8 15

6.2.1 Formulation du modèle 16

6.3 Exercice 9 17

6.3.1 Principe de l'algorithme EM 17

6.4 Exercice 10 18

6.4.1 Modèle de mélange gaussien 18

6.4.2 Estimation directe par optimisation 20

6.4.3 Analyse de performance selon l'annexe 20

6.5 Exercice 11 21

6.5.1 Modèle et critère d'ajustement 21

6.6 Exercice 12 24

6.6.1 Modèle logistique sans intercept 24

6.7 Conclusion générale 26

1 Projet d'optimisation numérique

1.1 Rapport Python

1.1.1 Auteurs

- Bocoum Amadou
- Anya Levêque

Ce rapport étudie plusieurs problèmes d'optimisation issus à la fois de l'analyse numérique et de la statistique. L'objectif est double : d'une part, mettre en oeuvre plusieurs familles d'algorithmes d'optimisation ; d'autre part, comparer leur comportement sur des problèmes de structures différentes, en termes d'efficacité numérique et de robustesse.

Les méthodes considérées incluent notamment la descente de gradient, Newton-Raphson, Gauss-Newton, Levenberg-Marquardt, EM, ainsi que plusieurs méthodes d'optimisation directe. Chaque algorithme est appliqué dans un cadre adapté, en utilisant soit des dérivées analytiques, soit des approximations numériques lorsque cela est nécessaire.

Une place particulière est accordée à la fonction de Rosenbrock, qui constitue un problème test classique en optimisation continue. Sa vallée étroite et sa géométrie fortement anisotrope en font un cadre particulièrement instructif pour analyser les différences de comportement entre méthodes de descente, méthodes quasi-Newton et méthodes de second ordre.

Pour comparer les méthodes, nous utilisons la méthodologie proposée dans l'annexe du sujet. Celle-ci consiste à considérer plusieurs points initiaux comme des problèmes différents, à mesurer le temps d'exécution de chaque algorithme, puis à construire des profils de performance permettant de comparer leur comportement global.

L'enjeu n'est donc pas seulement d'obtenir une solution numérique, mais aussi de comprendre comment la nature du problème influence le choix de la méthode, ainsi que les compromis observés entre vitesse de convergence, précision et robustesse.

Au fil du projet, nous apprenons ainsi à :

Compétence travaillée	Illustration dans le projet
Formaliser un problème d'optimisation	fonctions de Rosenbrock, vraisemblances, sommes de carrés
Dériver les outils analytiques utiles	gradient, Hessienne, score, information de Fisher
Mettre en oeuvre des algorithmes spécialisés	EM, Gauss-Newton, Levenberg-Marquardt, Fisher
Comparer les méthodes de manière rigoureuse	profils de performance construits selon l'annexe
Interpréter un résultat numérique	efficacité, robustesse, sensibilité aux initialisations

Dans toutes les parties où plusieurs algorithmes sont comparés, nous suivons **strictement la méthodologie d'analyse de performance de l'annexe**. Le coût attribué à l'algorithme α sur le problème p est noté $\tau_{p,\alpha}$ et vaut le temps d'exécution lorsque le problème est effectivement résolu ; sinon, on pose $\tau_{p,\alpha} = +\infty$.

1.1.2 Rappel mathématique : méthodologie de l'annexe

Pour chaque problème p et chaque algorithme α, nous considérons :

$$\tau_{p,\alpha} = \begin{cases} \text{temps d'exécution de } \alpha \text{ sur } p, & \text{si } \alpha \text{ résout } p, \\ +\infty, & \text{sinon.} \end{cases}$$

On définit ensuite la meilleure performance observée sur le problème p par

$$T_p = \min_{\alpha} \tau_{p,\alpha},$$

puis le ratio de performance normalisé :

$$\rho_{p,\alpha} = \frac{\tau_{p,\alpha}}{T_p}.$$

Enfin, le profil de performance de l'algorithme α est la fonction

$$P_\alpha(\pi) = \frac{1}{|\mathcal{P}|} \#\{p \in \mathcal{P} : \rho_{p,\alpha} \leq \pi\}.$$

Cette construction permet une lecture immédiate :

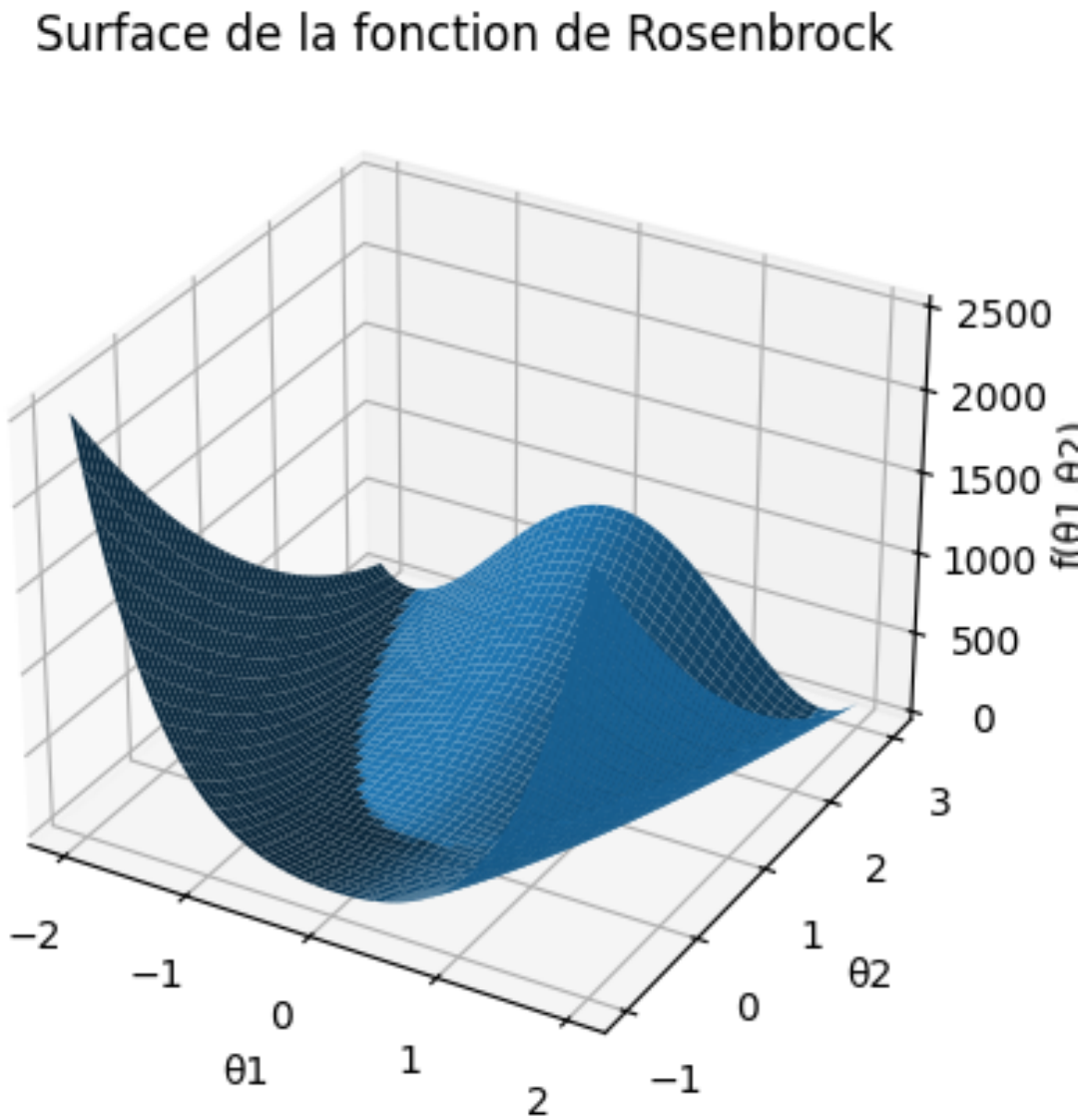
Quantité	Interprétation
$P_\alpha(1)$	proportion de problèmes sur lesquels α est le meilleur
π proche de 1	zone d'**efficacité pure**
π proche de R	zone de **robustesse**
R	borne choisie strictement au-dessus des ratios finis pour coder les échecs

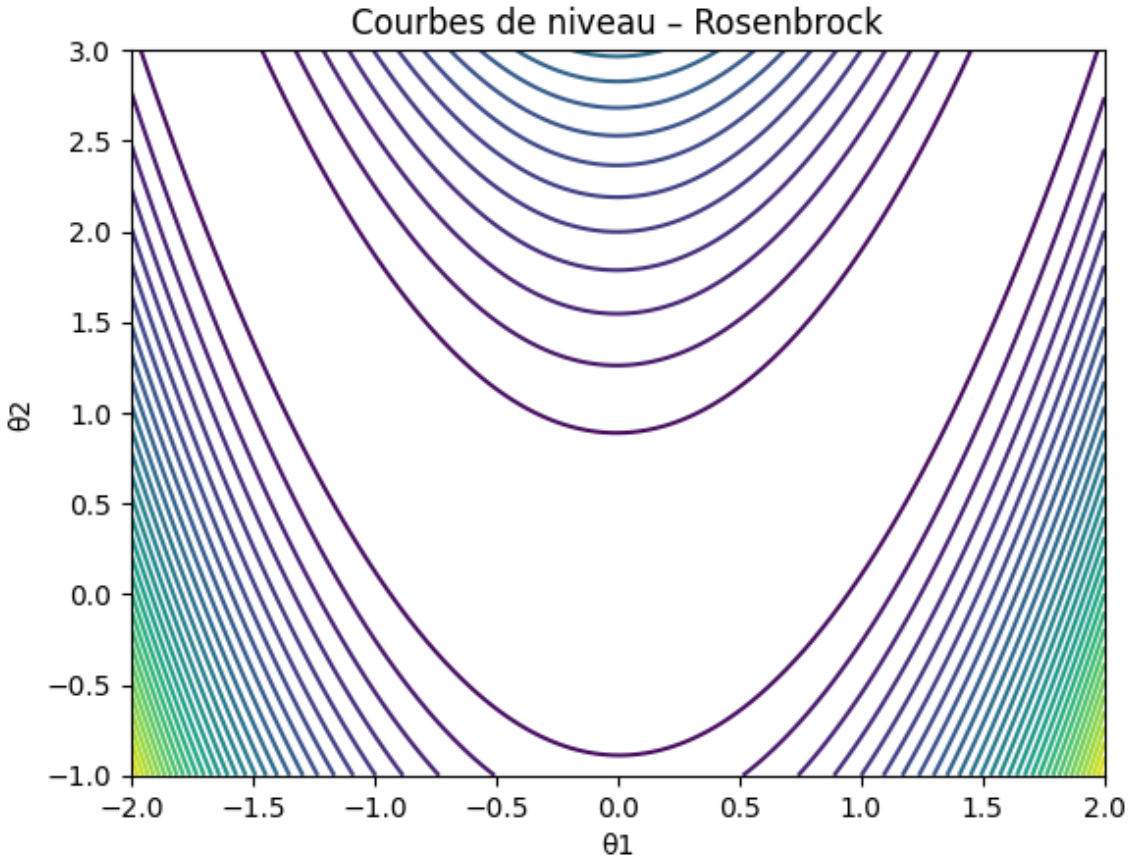
Ainsi, les figures de performance ne comparent pas seulement des temps moyens : elles distinguent **la vitesse quand l'algorithme réussit** et **sa capacité à réussir sur toute une famille de problèmes**.

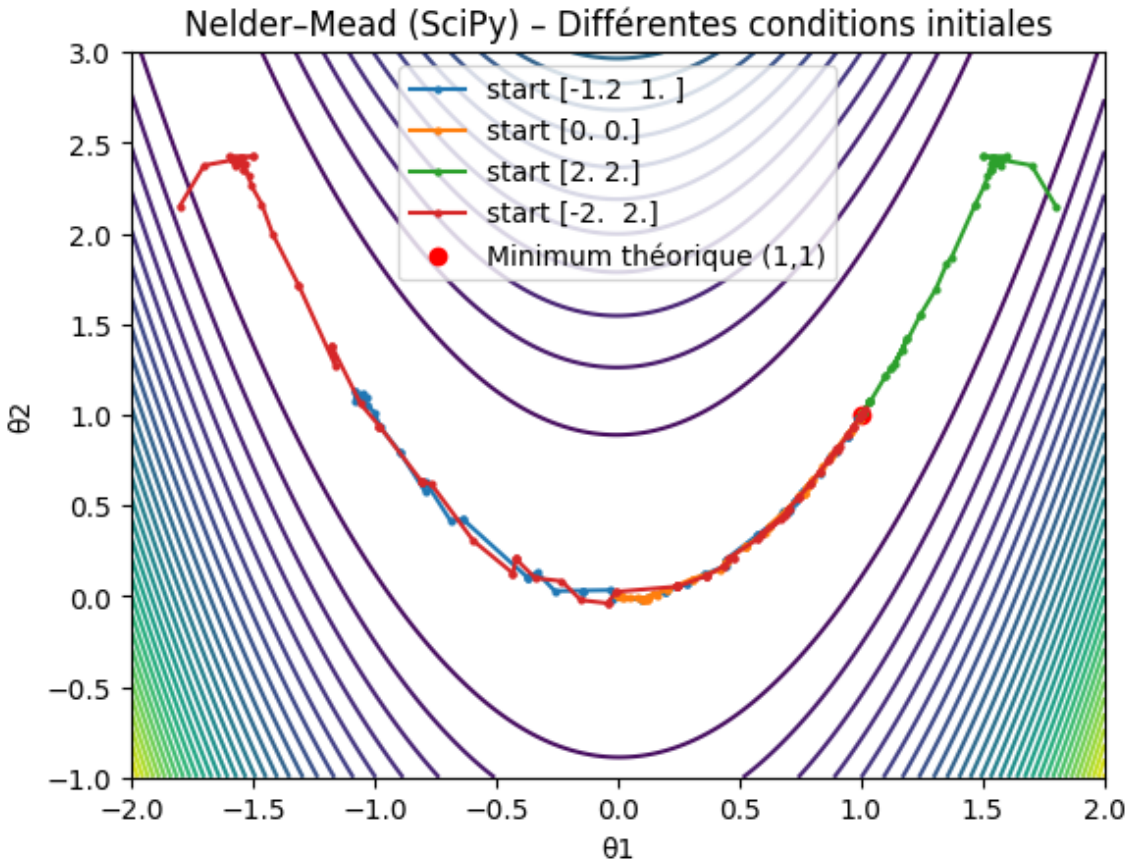
1.2 Exercice 1

On étudie ici la fonction de Rosenbrock, puis on applique l'algorithme de Nelder-Mead à partir de plusieurs conditions initiales afin d'observer les trajectoires de convergence vers le minimum global.

Condition initiale	**Solution trouvée**	$f(x^*)$	**Itér.**	**Éval.**	**Conv.**
$[-1.2,\ 1.]$	$[1.00002202,\ 1.00004222]$	$8.17766119741667485 \cdot 10^{-10}$	85	159	True
$[0.,\ 0.]$	$[1.00000439,\ 1.00001064]$	$3.68617691517590779 \cdot 10^{-10}$	79	146	True
$[2.,\ 2.]$	$[0.99998292,\ 0.99996512]$	$3.42395529250897662 \cdot 10^{-10}$	62	119	True
$[-2.,\ 2.]$	$[0.99999085,\ 0.99998346]$	$3.92431208003859761 \cdot 10^{-10}$	112	208	True


Surface de la fonction de Rosenbrock
2500
2000
1500
1000
500
0
f(θ1, θ2)
−2
−1
0
1
2
θ1
−1
0
1
2
3
θ2


Courbes de niveau – Rosenbrock
θ2
θ1
3.0
2.5
2.0
1.5
1.0
0.5
0.0
−0.5
−1.0
−2.0
−1.5
−1.0
−0.5
0.0
0.5
1.0
1.5
2.0


Nelder–Mead (SciPy) – Différentes conditions initiales
start [-1.2 1.]
start [0. 0.]
start [2. 2.]
start [-2. 2.]
Minimum théorique (1,1)
θ2
θ1
3.0
2.5
2.0
1.5
1.0
0.5
0.0
−0.5
−1.0
−2.0
−1.5
−1.0
−0.5
0.0
0.5
1.0
1.5
2.0

Observations: Nous avons appliqué l'algorithme de Nelder-Mead à la fonction de Rosenbrock, connue pour présenter une vallée étroite rendant l'optimisation difficile.

Plusieurs points initiaux ont été testés afin d'évaluer la robustesse de l'algorithme. Dans tous les cas, la convergence vers le minimum global (1,1) est observée avec une grande précision.

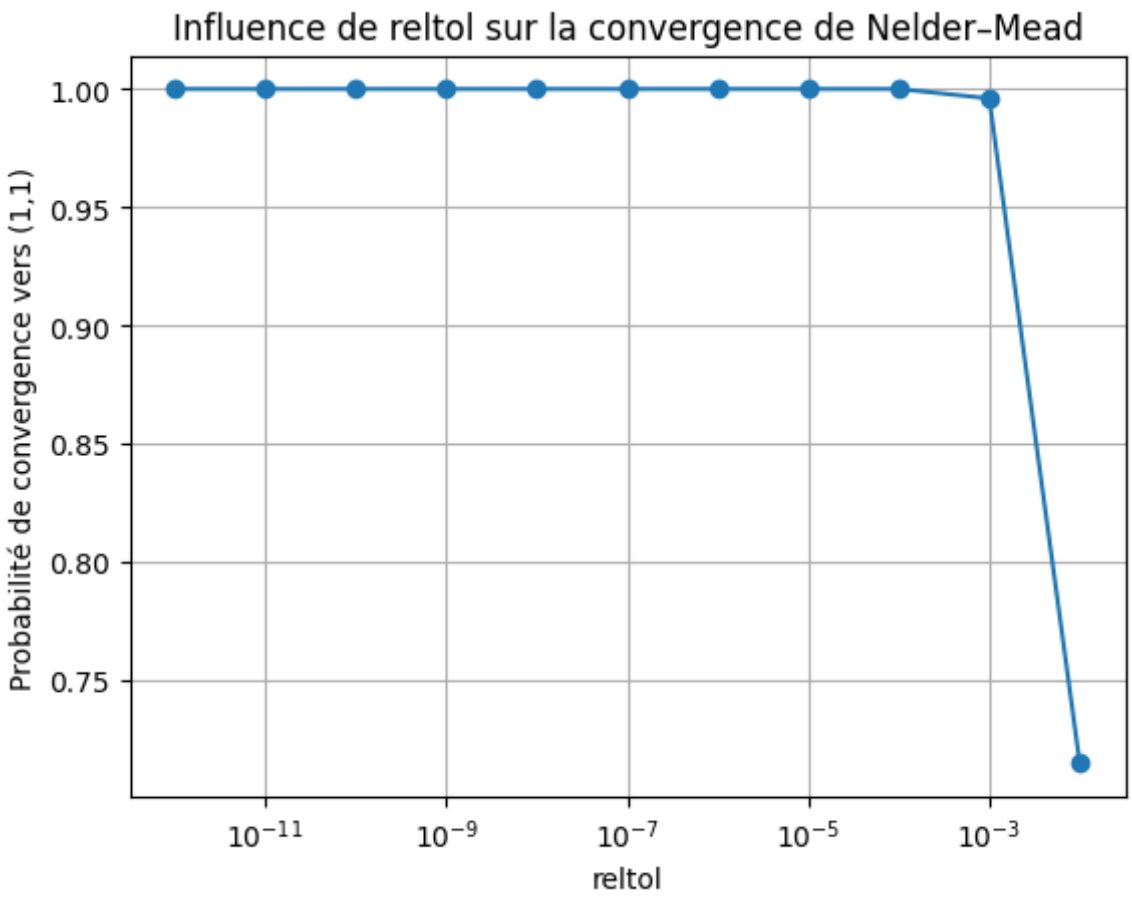
On constate néanmoins une dépendance du nombre d'itérations vis-à-vis du point initial : certains points nécessitent davantage d'itérations, ce qui illustre la difficulté du problème.

Les trajectoires montrent que l'algorithme suit la vallée de la fonction, ce qui explique la convergence relativement lente typique de la méthode Nelder-Mead sur ce type de problème.

1.3 Exercice 2

On étudie ensuite l'influence du paramètre de tolérance `reltol` sur la probabilité de convergence de Nelder-Mead vers le minimum global de la fonction de Rosenbrock.

Information	**Valeur**
Probabilité de convergence vers (1,1)	1.0



Nous avons étudié l'influence du paramètre de tolérance reltol sur la probabilité de convergence de l'algorithme de Nelder-Mead vers le minimum global de la fonction de Rosenbrock.

Pour cela, 1000 conditions initiales ont été générées uniformément sur le domaine [−2,2] 2 . Pour chaque valeur de reltol, l'algorithme est exécuté et nous vérifions si la solution obtenue est suffisamment proche du minimum global (1,1).

Les résultats montrent que lorsque la tolérance est très faible, la probabilité de convergence est proche de 1. En revanche, lorsque la tolérance augmente, l'algorithme s'arrête plus tôt, ce qui diminue la précision de la solution et donc la probabilité de convergence vers le minimum global.

Cela illustre le compromis classique entre précision et temps de calcul dans les méthodes d'optimisation.

On remarque également que la transition entre convergence parfaite et perte de performance est relativement abrupte, ce qui met en évidence une forte sensibilité du critère d'arrêt dans ce problème.

1.4 Exercices 3 à 5

Dans cette partie, on compare progressivement plusieurs algorithmes sur la fonction de Rosenbrock :

- `Nelder-Mead`
- `Plus grande pente`
- `Newton-Raphson`
- `BFGS`
- `CG`

Les profils de performance sont construits selon la méthodologie de l'annexe, et l'on montre l'effet de l'ajout successif des méthodes au fil des exercices.

1.4.1 Guide de lecture des exercices 2 à 5

La comparaison sur la fonction de Rosenbrock est construite progressivement. Les méthodes ajoutées au fil des exercices n'exploitent pas les mêmes informations :

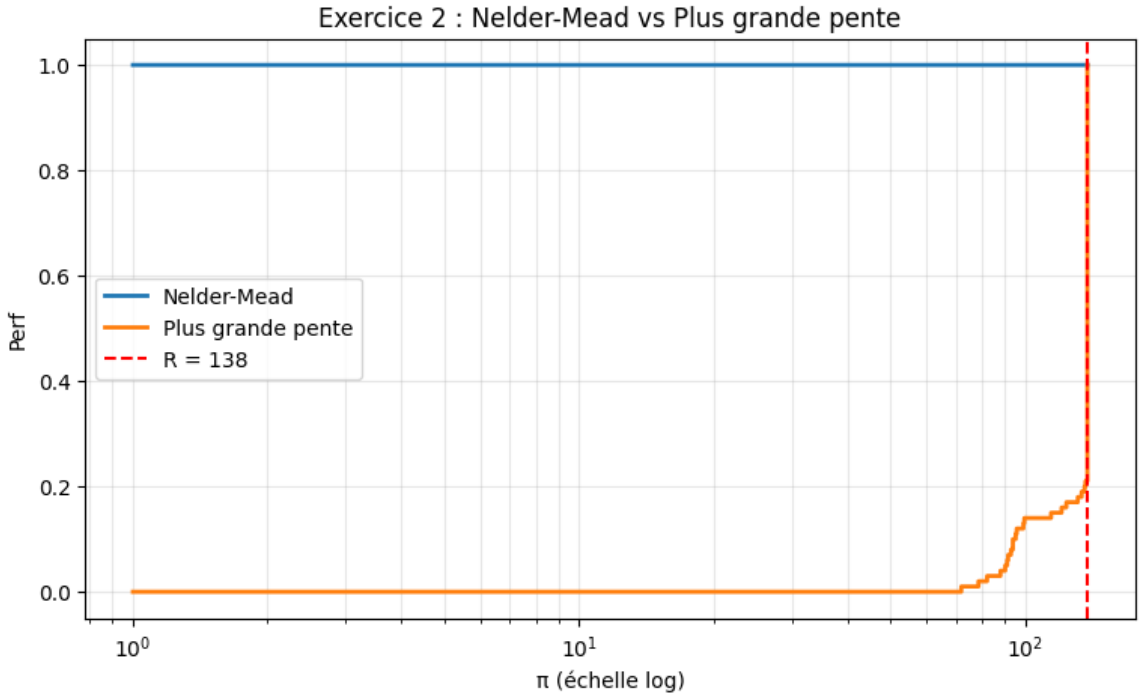
Algorithme	Nature	Information utilisée	Attendu théorique
`Nelder-Mead`	recherche directe	aucune dérivée	robuste en faible dimension, parfois plus lent
`Plus grande pente`	descente de gradient	gradient	simple mais souvent pénalisé sur les vallées étroites
`Newton-Raphson`	méthode de second ordre	gradient + Hessienne	très rapide près du minimum si la courbure est bien captée
`BFGS`	quasi-Newton	gradient	bon compromis entre vitesse et robustesse
`CG`	gradients conjugués	gradient	efficace sur certains problèmes lisses de grande taille

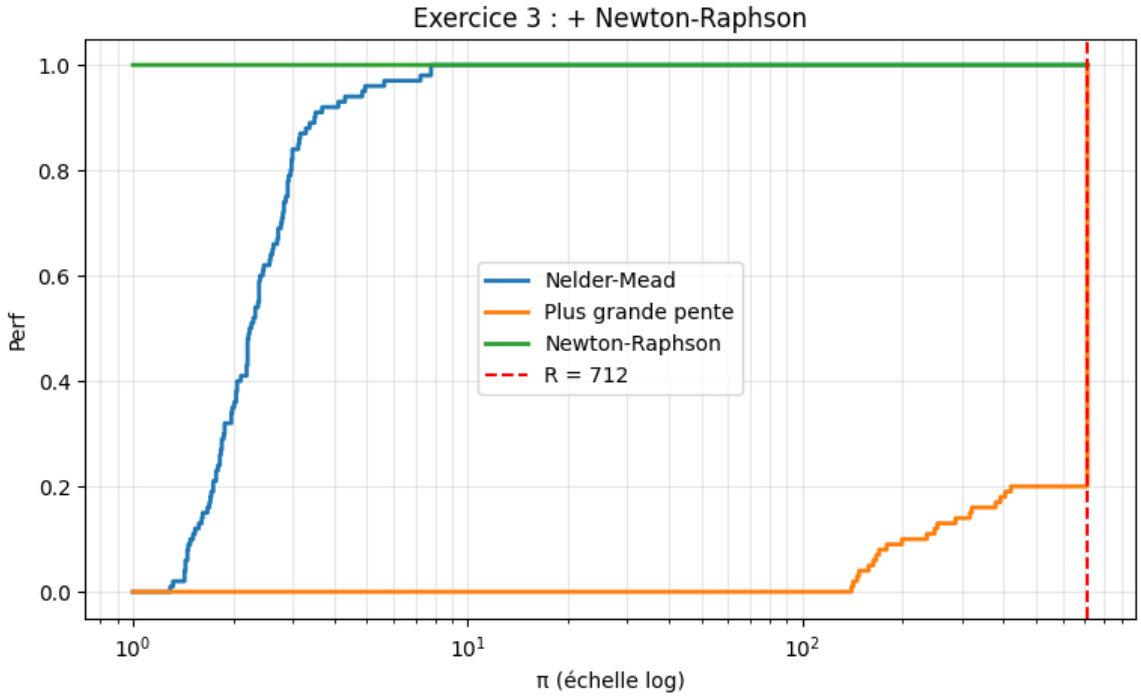
Algorithme	**Taux de succès**	**Moyenne**	**Médiane**	**Minimum**	**Maximum**
Nelder-Mead	100.00%	0.003285s	0.003260s	0.002144s	0.005254s
Plus grande pente	21.00%	0.303589s	0.301041s	0.295728s	0.331824s

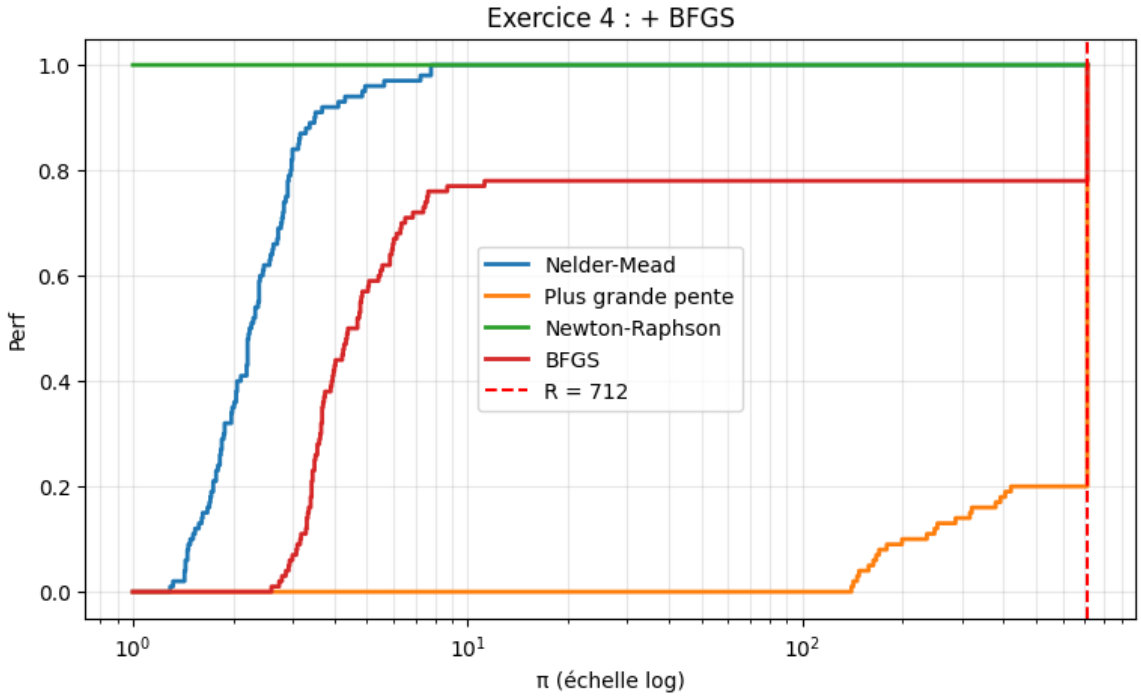
Algorithme	**Taux de succès**	**Moyenne**	**Médiane**	**Minimum**	**Maximum**
Nelder-Mead	100.00%	0.003285s	0.003260s	0.002144s	0.005254s
Plus grande pente	21.00%	0.303589s	0.301041s	0.295728s	0.331824s
Newton-Raphson	100.00%	0.001486s	0.001475s	0.000341s	0.002445s

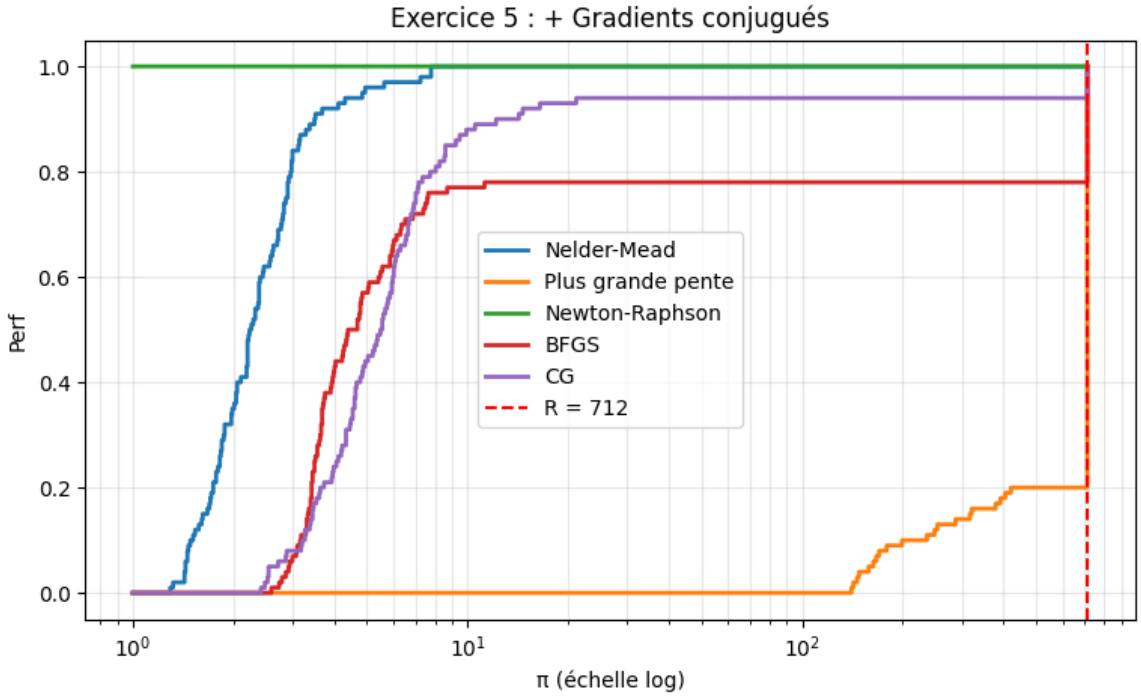
Algorithme	**Taux de succès**	**Moyenne**	**Médiane**	**Minimum**	**Maximum**
Nelder-Mead	100.00%	0.003285s	0.003260s	0.002144s	0.005254s
Plus grande pente	21.00%	0.303589s	0.301041s	0.295728s	0.331824s
Newton-Raphson	100.00%	0.001486s	0.001475s	0.000341s	0.002445s
BFGS	78.00%	0.006335s	0.006252s	0.002596s	0.011460s

Algorithme	**Taux de succès**	**Moyenne**	**Médiane**	**Minimum**	**Maximum**
Nelder-Mead	100.00%	0.003285s	0.003260s	0.002144s	0.005254s
Plus grande pente	21.00%	0.303589s	0.301041s	0.295728s	0.331824s
Newton-Raphson	100.00%	0.001486s	0.001475s	0.000341s	0.002445s
BFGS	78.00%	0.006335s	0.006252s	0.002596s	0.011460s
CG	94.00%	0.007594s	0.007202s	0.003304s	0.017587s


Exercice 2 : Nelder-Mead vs Plus grande pente
Nelder-Mead
Plus grande pente
R = 138
Perf
π (échelle log)


Exercice 3 : + Newton-Raphson
Nelder-Mead
Plus grande pente
Newton-Raphson
R = 712
Perf
π (échelle log)





2 Exercice 2:

Le profil de performance comparant Nelder-Mead et la méthode de la plus grande pente met en évidence une supériorité très nette de Nelder-Mead. Sa courbe vaut 1 dès $= 1$, ce qui signifie qu'il est systématiquement la méthode la plus performante sur l'ensemble des problèmes testés.

À l'inverse, la méthode de la plus grande pente présente un taux de succès beaucoup plus faible (21%) ainsi qu'un coût moyen très supérieur. Ce résultat montre que, sur la fonction de Rosenbrock, la méthode de la plus grande pente rencontre de fortes difficultés, ce qui s'explique par la présence

d'une vallée étroite et mal conditionnée.

3 Exercice 3:

L'ajout de la méthode de Newton-Raphson modifie fortement la hiérarchie des performances. Le profil obtenu montre que cette méthode domine désormais l'ensemble des autres, avec un taux de succès de 100% et un coût moyen plus faible que celui de Nelder-Mead.

Nelder-Mead reste une méthode robuste et performante, mais elle apparaît légèrement moins efficace que Newton-Raphson. La méthode de la plus grande pente, quant à elle, demeure très inférieure aux deux autres, tant en efficacité qu'en robustesse.

4 Exercice 4:

Avec l'introduction de BFGS, le profil de performance fait apparaître une méthode intermédiaire entre les approches les plus efficaces et les moins performantes. Newton-Raphson reste la meilleure méthode, tandis que Nelder-Mead conserve également de très bonnes performances.

BFGS présente un comportement correct, avec un taux de succès de 78%, mais reste moins robuste et moins efficace que Newton-Raphson et Nelder-Mead dans cette expérience. La méthode de la plus grande pente continue à être nettement dominée.

5 Exercice 5:

L'ajout de la méthode des gradients conjugués complète la comparaison entre les cinq algorithmes. Le profil de performance met en évidence la domination de Newton-Raphson, qui combine ici efficacité et robustesse. Nelder-Mead reste également très performant, avec un taux de succès de 100%, mais un coût moyen plus élevé que celui de Newton-Raphson.

La méthode CG apparaît globalement plus robuste que BFGS, avec un taux de succès de 94% contre 78%, et un comportement intermédiaire sur le profil. BFGS demeure compétitive sur une partie des problèmes, mais reste moins régulière. Enfin, la méthode de la plus grande pente est très nettement la moins performante, ce qui confirme qu'elle est mal adaptée à la géométrie de la fonction de Rosenbrock.

6 Synthèse:

L'analyse progressive des profils de performance permet de suivre l'effet de l'ajout successif des algorithmes dans les exercices 2 à 5. On observe que la méthode de Newton-Raphson domine globalement la comparaison finale, tant en efficacité qu'en robustesse. Nelder-Mead constitue une très bonne alternative, tandis que CG et BFGS présentent des performances intermédiaires. La méthode de la plus grande pente apparaît enfin comme la moins adaptée à ce problème, ce qui est cohérent avec la difficulté classique de la fonction de Rosenbrock pour les méthodes de descente simples.

6.1 Exercices 6 et 7

L'exercice 6 porte sur l'estimation du paramètre d'une loi de Poisson tronquée par la méthode de Newton-Raphson. L'exercice 7 reprend le même problème en utilisant une méthode de recherche directe univariée, analogue à `optimize()` sous R. Enfin, les deux approches sont comparées à l'aide d'un profil de performance construit selon l'annexe.

6.1.1 Modèle statistique et séparation des exercices 6 et 7

Dans ces deux exercices, on travaille sur **le même modèle**, à savoir une loi de Poisson tronquée en zéro :

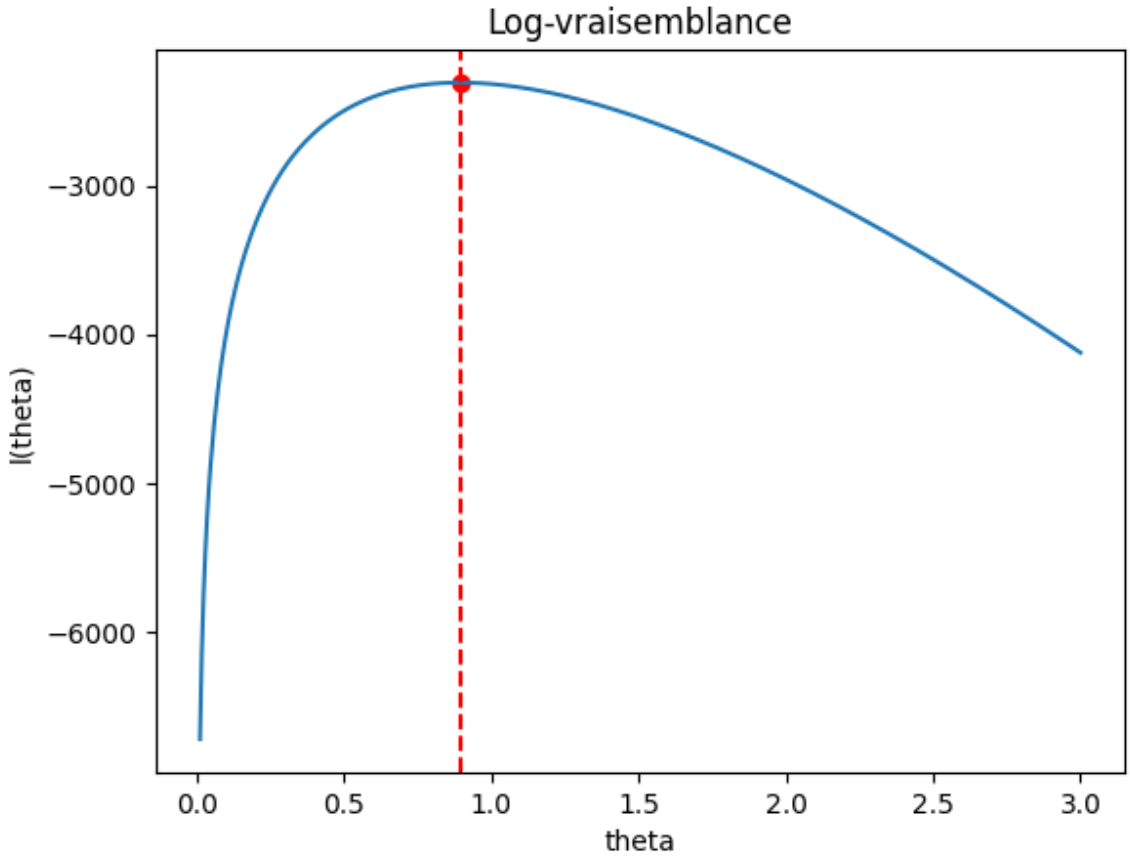
$$\mathbb{P}(X = x \mid \theta) = \frac{e^{-\theta}\theta^x/x!}{1 - e^{-\theta}}, \qquad x \geq 1.$$

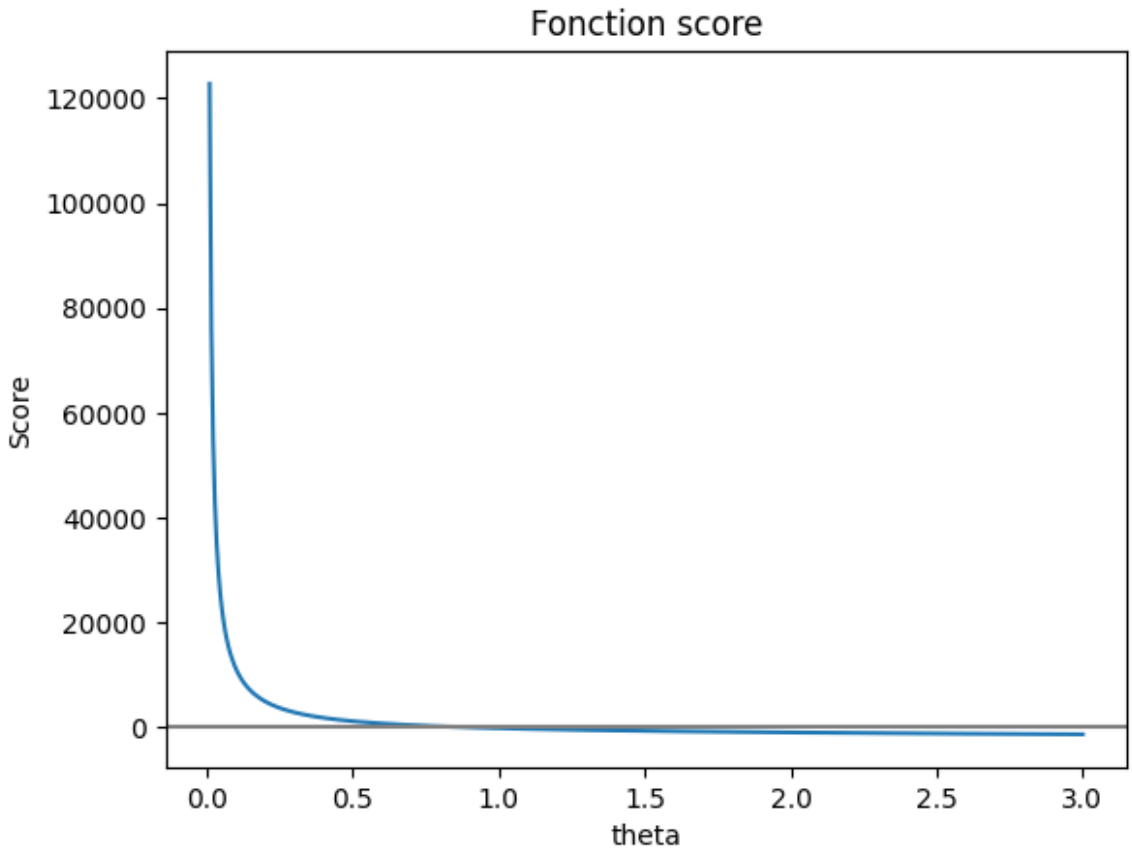
La différence porte sur la méthode numérique utilisée :

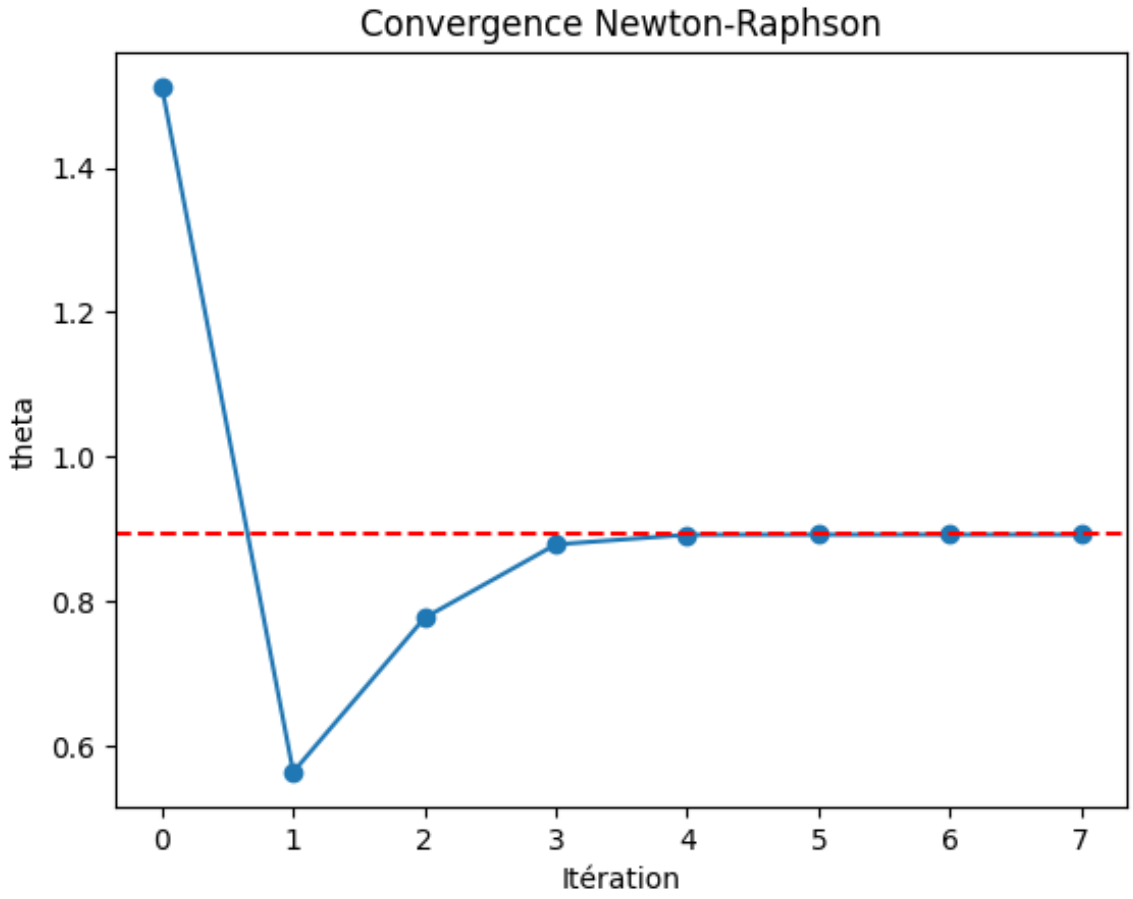
Exercice	Méthode	Idée
Exercice 6	`Newton-Raphson`	exploiter le score et la dérivée seconde de la log-vraisemblance
Exercice 7	optimisation directe univariée	résoudre le même problème sans dérivées explicites

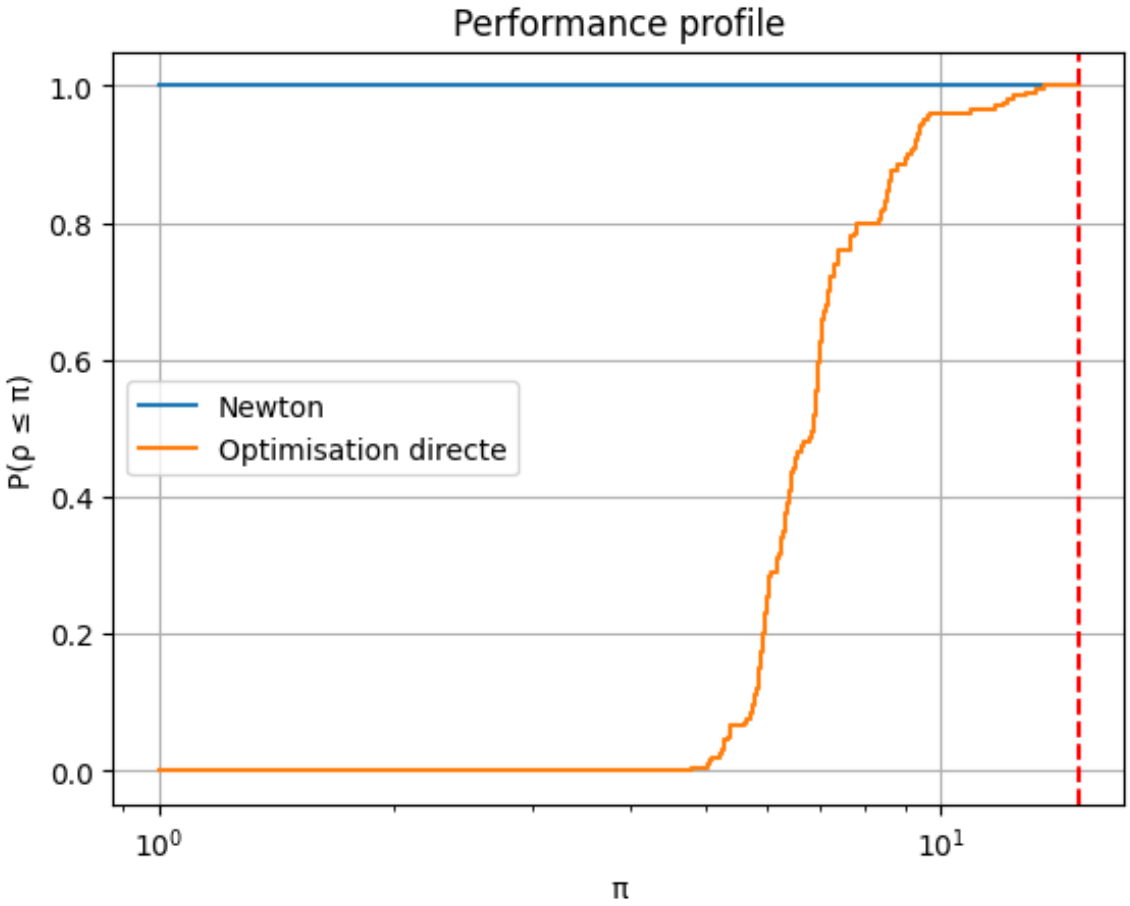
Le rapport distingue donc clairement **le problème statistique** et **la stratégie numérique** retenue pour le résoudre.

Information	**Valeur**
Estimateur Newton	0.8924960552804253
Estimateur optimize	0.8924965859929258


Log-vraisemblance
−3000
−4000
−5000
−6000
l(theta)
0.0
0.5
1.0
1.5
2.0
2.5
3.0
theta


Fonction score
120000
100000
80000
60000
40000
20000
0
Score
0.0
0.5
1.0
1.5
2.0
2.5
3.0
theta


Convergence Newton-Raphson
theta
1.4
1.2
1.0
0.8
0.6
0
1
2
3
4
5
6
7
Itération



Le profil de performance obtenu montre que la méthode de Newton-Raphson est la plus efficace sur l'ensemble des problèmes considérés. En effet, sa courbe atteint immédiatement la valeur 1 pour = 1, ce qui signifie qu'elle fournit systématiquement la meilleure performance relative parmi les méthodes comparées.

La méthode d'optimisation directe présente des performances plus faibles. Sa courbe reste d'abord proche de 0, puis augmente progressivement lorsque devient plus grand. Cela signifie qu'elle nécessite en général un coût plus important que Newton-Raphson pour résoudre un même problème.

Les deux méthodes atteignent néanmoins la valeur 1 pour des valeurs suffisamment grandes de , ce qui indique qu'elles parviennent toutes deux à résoudre les problèmes testés. La différence observée concerne donc principalement l'efficacité et non la robustesse.

Ce comportement est cohérent avec la nature du problème étudié. La méthode de Newton-Raphson exploite l'information dérivée disponible dans le modèle, ce qui accélère fortement la convergence. À l'inverse, l'optimisation directe repose sur une stratégie plus générale, mais moins rapide dans ce contexte.

6.2 Exercice 8

On considère ici l'estimation des paramètres d'une loi gamma décalée. Après reparamétrisation pour traiter les contraintes du modèle, on compare `Nelder-Mead` et `BFGS` sur une famille de conditions initiales en appliquant la méthodologie de l'annexe.

6.2.1 Formulation du modèle

Dans l'exercice 8, on considère un modèle gamma décalé de la forme

$$X = \alpha + Y, \qquad Y \sim \Gamma(p, \sigma),$$

avec les contraintes

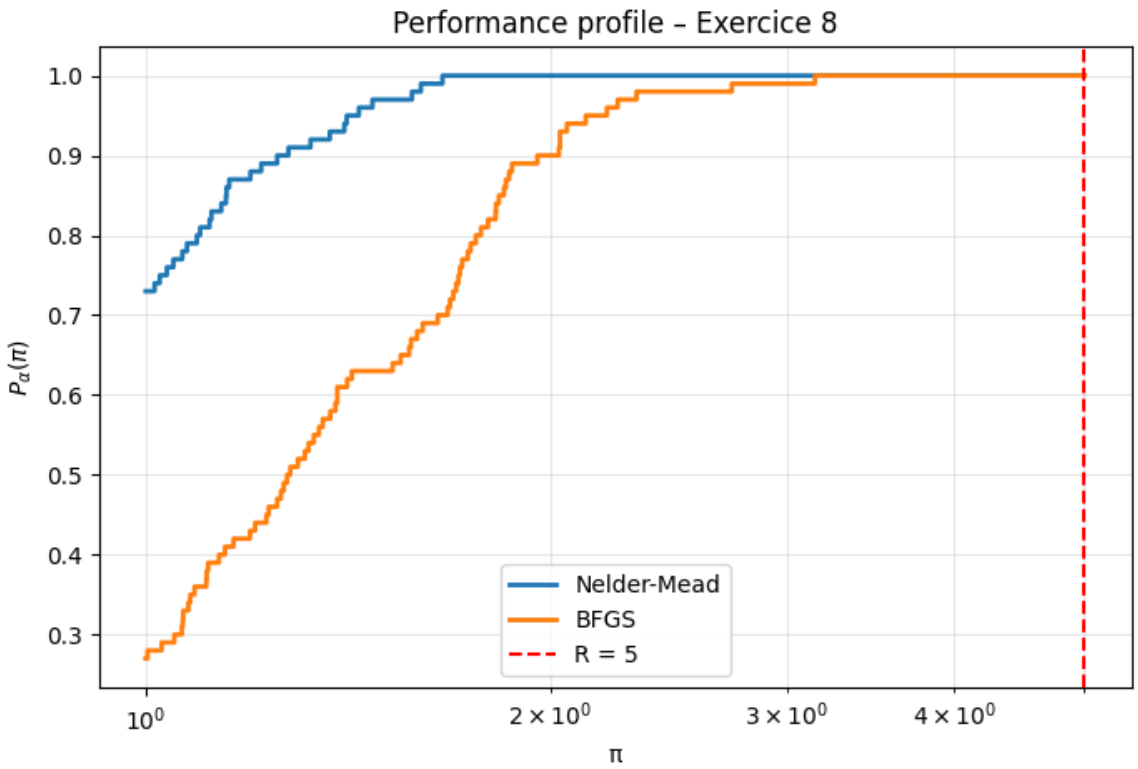
$$\alpha < \min_i x_i, \qquad \sigma > 0, \qquad p > 2.$$

La reparamétrisation utilisée dans le code permet de transformer ce problème contraint en un problème d'optimisation sans contraintes, ce qui facilite l'application de `Nelder-Mead` et de `BFGS`.

```
xmin = 5.6421950015636915

Estimation par Nelder-Mead (énoncé) :
alpha_hat = 4.870150395485001
sigma_hat = 0.9689242673909327
p_hat     = 6.380916626168563
negloglik = 11298.52899892534
```

Algorithme	**Taux de succès**	**Moyenne**	**Médiane**	**Minimum**	**Maximum**
Nelder-Mead	100.00%	0.016062s	0.015838s	0.012162s	0.022039s
BFGS	100.00%	0.021418s	0.020428s	0.010049s	0.045574s



Le profil de performance met en évidence une supériorité globale de la méthode de Nelder–Mead sur l'ensemble des problèmes considérés. En effet, sa courbe domine strictement celle de BFGS pour

la majorité des valeurs de , ce qui indique que Nelder–Mead est plus fréquemment proche de la meilleure performance observée.

Plus précisément, la valeur de P (1) montre que Nelder–Mead est l'algorithme le plus performant sur une proportion significativement plus élevée de problèmes. Cela suggère que, pour de nombreuses initialisations, il atteint une solution de qualité comparable en un temps plus faible que BFGS.

Toutefois, les deux courbes convergent vers 1 lorsque augmente, ce qui indique que les deux méthodes sont globalement robustes et parviennent à résoudre la quasi-totalité des problèmes. La différence entre les algorithmes réside donc principalement dans leur efficacité relative plutôt que dans leur capacité à converger.

Ce comportement peut s'expliquer par la structure du problème et la reparamétrisation utilisée. Bien que BFGS soit une méthode à gradient supposée plus efficace sur des fonctions lisses, l'absence de gradient analytique (SciPy utilise un gradient numérique par défaut) dans cette implémentation implique l'utilisation d'approximation numérique, ce qui introduit un surcoût et peut dégrader ses performances. À l'inverse, Nelder–Mead, bien que dérivée-free, apparaît ici mieux adapté à la géométrie locale de la fonction objectif.

Ainsi, ce cas d'étude illustre qu'en pratique, les méthodes exploitant des dérivées ne sont pas systématiquement supérieures, en particulier lorsque celles-ci doivent être approchées numériquement.

6.3 Exercice 9

6.3.1 Principe de l'algorithme EM

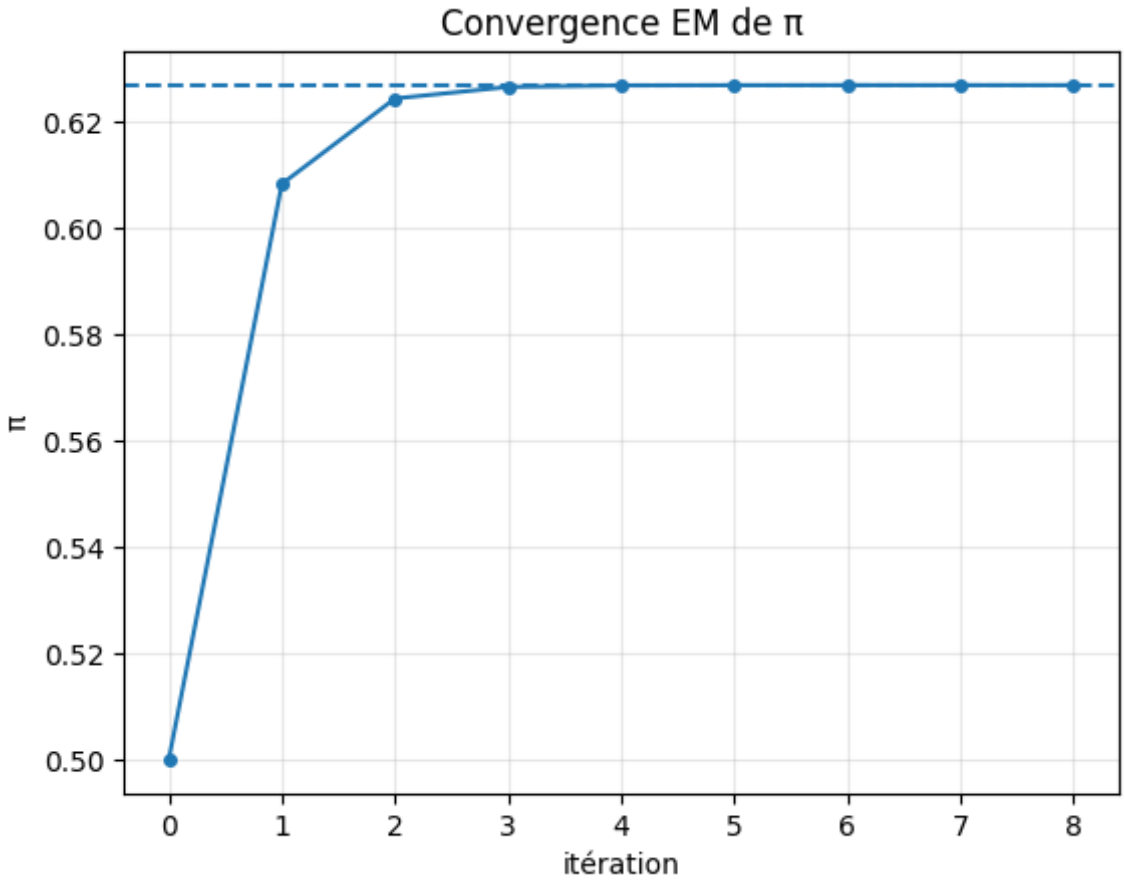
L'algorithme EM alterne deux étapes :

1. une étape **E** où l'on calcule l'espérance de la variable latente conditionnellement aux observations et au paramètre courant ;
2. une étape **M** où l'on maximise la vraisemblance complétée espérée.

Dans ce cadre, l'itération produit une suite $(\pi^{(k)})_{k\geq 0}$ telle que la vraisemblance ne décroît pas. La stabilisation rapide observée numériquement est donc cohérente avec la théorie d'EM sur un modèle de faible dimension.

L'exercice 9 traite un problème d'estimation avec données manquantes, résolu à l'aide de l'algorithme EM.

```
Convergence atteinte à l'itération 8
pi_hat = 0.626821484140
```



Nous avons appliqué l'algorithme EM afin d'estimer le paramètre à partir des données observées.

L'évolution de l'estimateur au cours des itérations est représentée sur la figure ci-dessus.

On observe que la convergence est très rapide, puisque la valeur de se stabilise en moins de 5 itérations. À partir de l'itération 3, les variations deviennent négligeables, et l'algorithme atteint le critère d'arrêt fixé.

La valeur finale obtenue est :

$\hat{}$ 0.627

Cette rapidité de convergence est caractéristique de l'algorithme EM dans des modèles de faible dimension, où les mises à jour sont explicites et bien conditionnées.

De plus, la trajectoire est monotone croissante, ce qui est cohérent avec la propriété fondamentale de l'algorithme EM : la vraisemblance ne décroît jamais au cours des itérations.

Enfin, la stabilisation rapide autour de la valeur finale indique que le problème ne présente pas de difficultés numériques particulières, et que l'initialisation choisie ((0)=0.5) est adaptée.

6.4 Exercice 10

6.4.1 Modèle de mélange gaussien

Le modèle étudié est un mélange de deux lois normales unidimensionnelles :

$$f(x) = p\,\varphi(x;\mu_1,\sigma_1) + (1-p)\,\varphi(x;\mu_2,\sigma_2),$$

où $0 < p < 1$ et $\varphi(\cdot; \mu, \sigma)$ désigne la densité gaussienne de moyenne μ et d'écart-type σ.

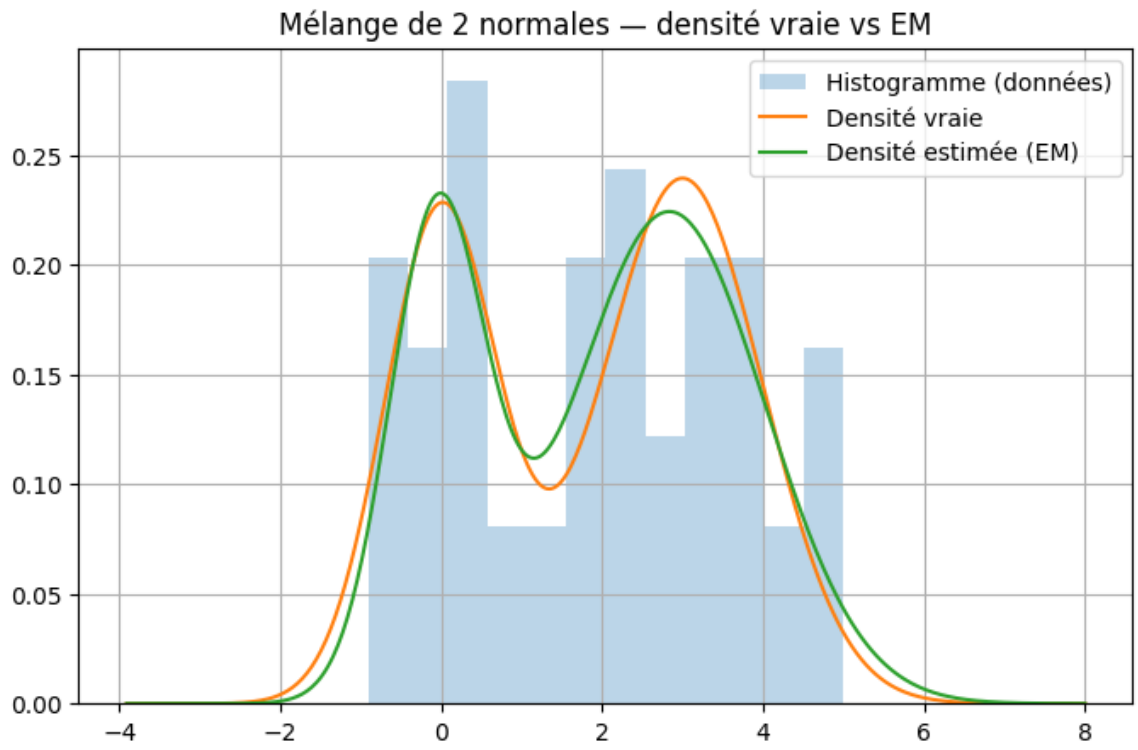
L'exercice 10 se lit en trois niveaux complémentaires :

Partie	Objectif
EM	exploiter la structure latente du modèle
optimisation directe	comparer des méthodes générales sur la même log-vraisemblance
profil de performance	comparer efficacité et robustesse sur plusieurs initialisations

Cet exercice porte sur l'estimation des paramètres d'un mélange de deux lois normales. On procède en trois temps :

1. estimation par EM ;
2. estimation directe par méthodes d'optimisation générale ;
3. comparaison des algorithmes selon la méthodologie de l'annexe.

Information	**Valeur**
EM convergence	True
EM itérations	49
EM theta	[p, mu1, mu2, sig1, sig2] = [0.32945334 -0.05059046 2.83570328 0.59720525 1.19317313]



L'algorithme EM converge correctement pour l'estimation des paramètres du mélange de deux lois normales. Dans notre exécution, la convergence est atteinte en 49 itérations, ce qui reste raisonnable pour un problème de mélange où les variables latentes ne sont pas observées directement.

Les paramètres estimés sont p 0.329, mu1 -0.051, mu2 2.836, sig1 0.597 et sig2 1.193. Ces valeurs sont globalement proches des paramètres vrais utilisés pour la simulation, à savoir p = 0.4, mu1 = 0, mu2 = 3, sig1 = sqrt(0.5) et sig2 = 1. On observe donc que l'algorithme EM restitue correctement la structure générale du mélange, même si les estimations ne coïncident pas exactement avec les valeurs théoriques, ce qui est normal compte tenu de la variabilité d'échantillonnage.

Le graphique comparant la densité vraie et la densité estimée confirme cette bonne qualité d'ajustement. La courbe estimée par EM suit globalement la forme bimodale de la densité réelle et reproduit correctement la position des deux composantes du mélange. Les écarts observés restent modérés et peuvent s'expliquer par la taille finie de l'échantillon ainsi que par la sensibilité habituelle des modèles de mélange aux fluctuations des données.

Dans l'ensemble, cette première étape montre que l'algorithme EM constitue une méthode adaptée pour estimer les paramètres du modèle, avec une convergence stable et une qualité d'ajustement satisfaisante.

6.4.2 Estimation directe par optimisation

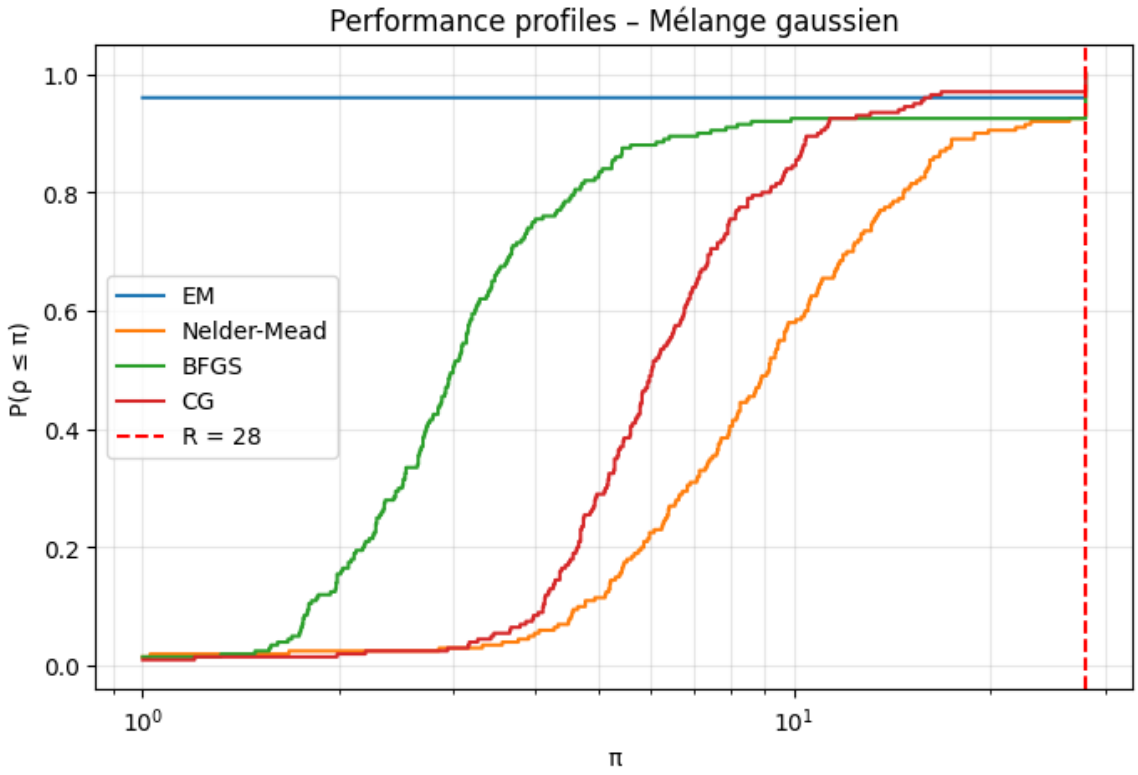
On compare ici les estimateurs obtenus par `Nelder-Mead`, `BFGS` et `CG` en minimisant directement la négative log-vraisemblance du mélange gaussien.

Information	**Valeur**
Résultats optim	{'Nelder-Mead': array([0.35418711, -0.01688992, 2.89645793, 0.6726614 , 0.91688422]), 'BFGS': array([0.35419387, -0.01685987, 2.89643657, 0.67269394, 0.91688437]), 'CG': array([0.35419385, -0.01685989, 2.89643655, 0.67269392, 0.91688438])}

6.4.3 Analyse de performance selon l'annexe

Afin de comparer objectivement `EM`, `Nelder-Mead`, `BFGS` et `CG`, on considère une famille de conditions initiales et on construit les profils de performance correspondants.

Algorithme	**Taux de succès**	**Moyenne**	**Médiane**	**Minimum**	**Maximum**
EM	96.00%	0.009519s	0.008127s	0.002570s	0.021956s
Nelder-Mead	92.50%	0.080817s	0.076967s	0.042397s	0.179713s
BFGS	92.50%	0.028243s	0.025330s	0.015284s	0.077650s
CG	97.00%	0.060384s	0.056211s	0.028572s	0.222137s



Le profil de performance met en évidence une supériorité nette de l'algorithme EM pour l'estimation des paramètres du mélange gaussien. Sa courbe domine l'ensemble des autres méthodes sur presque tout le domaine étudié, et sa valeur élevée dès $= 1$ montre qu'il est le plus souvent l'algorithme le plus performant sur les problèmes considérés. Ce résultat est cohérent avec la structure du problème, l'algorithme EM étant spécifiquement conçu pour les modèles de mélange.

Parmi les méthodes d'optimisation générale, l'algorithme BFGS apparaît comme la meilleure alternative à EM. Sa courbe est systématiquement au-dessus de celles de CG et de Nelder-Mead, ce qui indique une meilleure efficacité relative sur la majorité des conditions initiales testées. L'algorithme des gradients conjugués présente des performances intermédiaires, tandis que Nelder-Mead apparaît comme la méthode la moins efficace dans ce cadre expérimental.

Les taux de succès restent toutefois élevés pour l'ensemble des méthodes, avec des valeurs comprises entre 92.5% et 97%. Ainsi, les différences observées concernent davantage l'efficacité que la robustesse. En particulier, EM combine ici les deux avantages : un très bon taux de succès et un coût moyen nettement plus faible que celui des autres algorithmes.

Dans l'ensemble, cette analyse confirme que, pour le problème étudié, une méthode spécialisée comme EM est particulièrement adaptée. Les méthodes générales d'optimisation permettent également d'obtenir des solutions satisfaisantes, mais au prix d'un coût de calcul plus important et d'une performance globale moins régulière.

6.5 Exercice 11

6.5.1 Modèle et critère d'ajustement

L'exercice 11 porte sur le modèle non linéaire

$$y_i = \frac{\theta_1 x_i}{\theta_2 + x_i} + \varepsilon_i,$$

et l'ajustement consiste à minimiser la somme des carrés des résidus :

$$S(\theta_1, \theta_2) = \sum_{i=1}^{n} \left(y_i - \frac{\theta_1 x_i}{\theta_2 + x_i} \right)^2 .$$

Cette écriture met en évidence la différence entre les deux méthodes comparées :

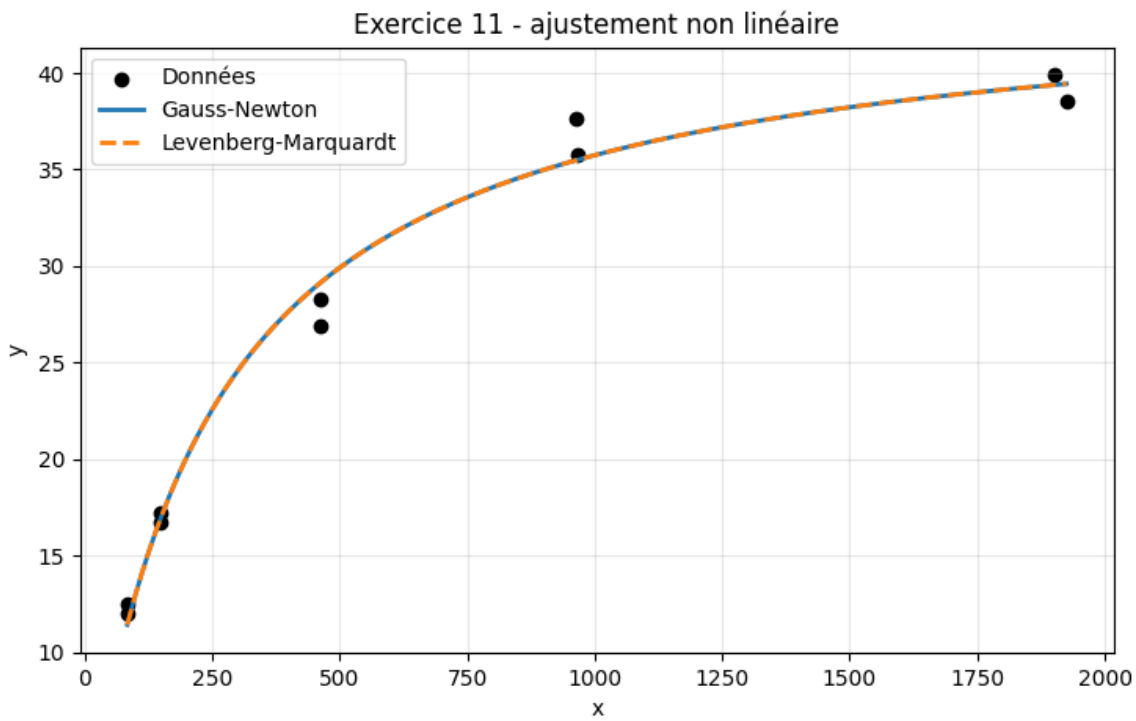
Méthode	Idée
`Gauss-Newton`	approximer localement le problème non linéaire par un problème quadratique
`Levenberg-Marquardt`	régulariser l'étape de Gauss-Newton pour gagner en robustesse

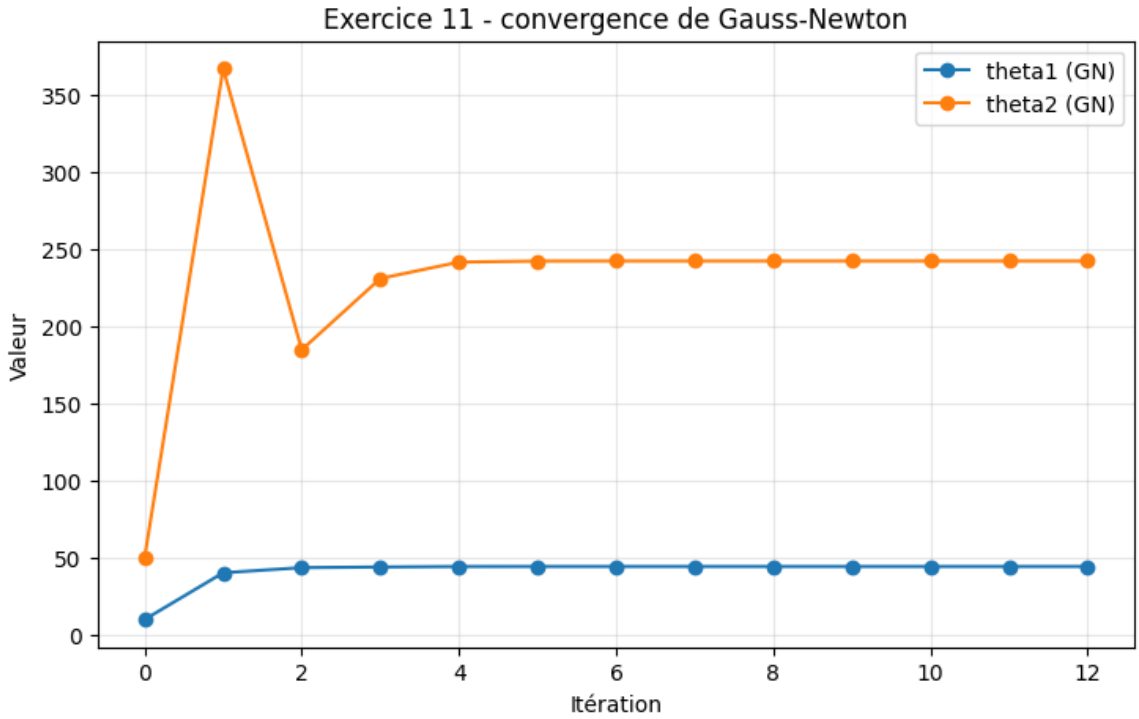
On ajuste le modèle non linéaire biochimique

```
y = theta1 * x / (theta2 + x)
```

d'abord par `Gauss-Newton`, puis par `Levenberg-Marquardt`. Dans un second temps, ces deux méthodes sont comparées à l'aide d'un profil de performance conforme à l'annexe.

Information	**Valeur**
Gauss-Newton	theta = [44.40167324 242.35505728], SSE = 13.15725157, itérations = 12
Levenberg-Marquardt	theta = [44.40167156 242.35501969], SSE = 13.15725157, itérations = 8

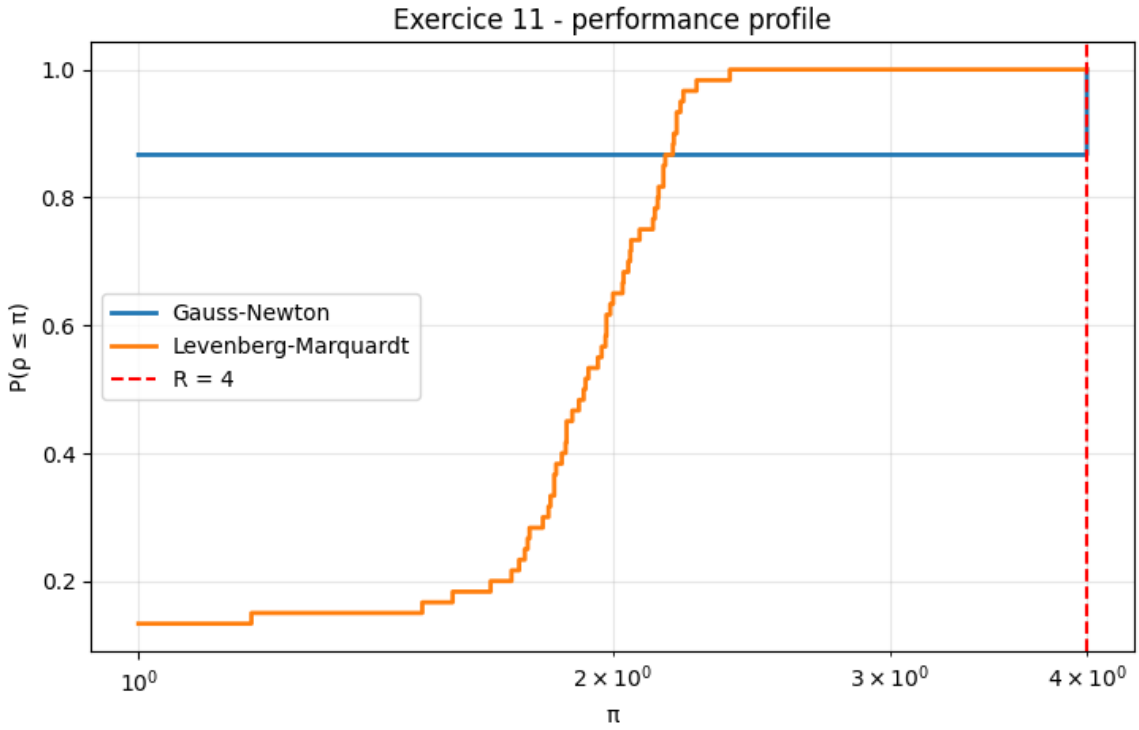




Les deux méthodes considérées, Gauss-Newton et Levenberg-Marquardt, conduisent à un ajustement pratiquement identique du modèle non linéaire aux données observées. Les courbes ajustées se superposent presque complètement, ce qui indique que les deux algorithmes atteignent la même solution d'estimation à partir des conditions initiales imposées.

La trajectoire de Gauss-Newton met en évidence une phase transitoire au début des itérations, en particulier pour le paramètre theta2, suivie d'une stabilisation rapide. Ce comportement montre que l'algorithme peut effectuer des corrections importantes au début de la procédure avant de se rapprocher de la solution finale. Dans l'ensemble, l'ajustement obtenu est satisfaisant et rend bien compte de l'évolution croissante et saturante des données expérimentales.

Algorithme	**Taux de succès**	**Moyenne**	**Médiane**	**Minimum**	**Maximum**
Gauss-Newton	86.67%	0.000573s	0.000496s	0.000369s	0.000926s
Levenberg-Marquardt	100.00%	0.001163s	0.001084s	0.000690s	0.002843s



Le profil de performance montre que Gauss-Newton est globalement la méthode la plus rapide lorsqu'elle converge, tandis que Levenberg-Marquardt est la plus robuste. En effet, Gauss-Newton présente des temps plus faibles mais un taux de succès de 86.67%, alors que Levenberg-Marquardt résout 100% des problèmes au prix d'un coût moyen légèrement supérieur. On retrouve ainsi un compromis net entre efficacité et robustesse.

6.6 Exercice 12

6.6.1 Modèle logistique sans intercept

L'exercice 12 considère un modèle logistique de la forme

$$\mathbb{P}(Y_i = 1 \mid X_i) = \frac{\exp(X_i^\top \beta)}{1 + \exp(X_i^\top \beta)},$$

où $X_i = (\text{âge}_i, \text{poids}_i)$ et $\beta = (\beta_1, \beta_2)$.

La négative log-vraisemblance minimisée dans le code est

$$\mathcal{L}(\beta) = \sum_{i=1}^{n} \left[\log\Big(1 + \exp(X_i^\top \beta)\Big) - y_i X_i^\top \beta \right].$$

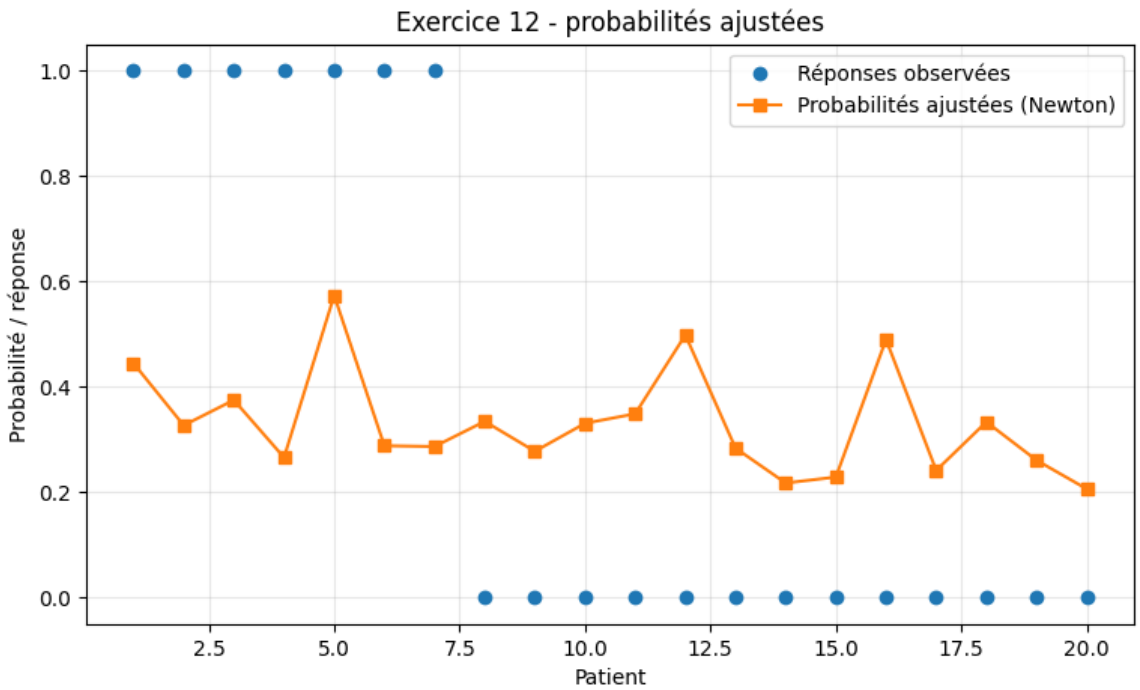
Le tableau suivant résume le statut des méthodes comparées :

Méthode	Information utilisée	Nature
`Newton-Raphson`	gradient + Hessienne	second ordre
`Fisher`	gradient + information de Fisher	second ordre régularisé par le modèle

Méthode	Information utilisée	Nature
`BFGS`	gradient	quasi-Newton
`Nelder-Mead`	aucune dérivée	recherche directe

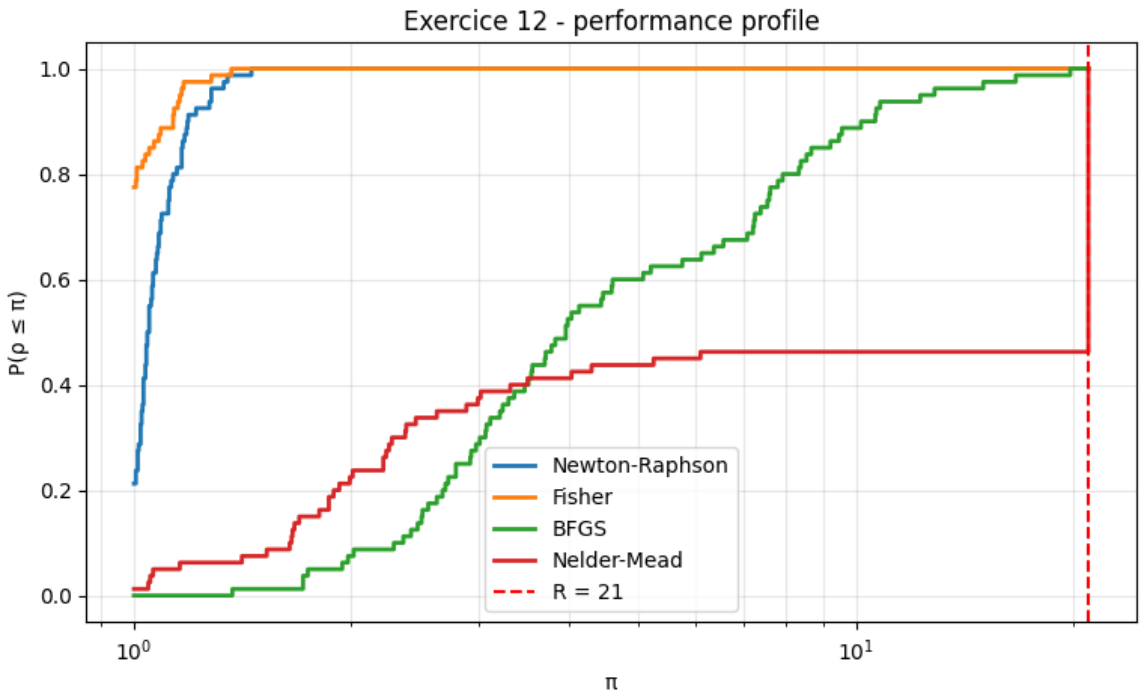
L'exercice 12 concerne l'ajustement d'un modèle de régression logistique sans intercept à partir des variables explicatives `âge` et `poids`. On compare d'abord les estimateurs obtenus par `Newton-Raphson`, la méthode du score de Fisher, `BFGS` et `Nelder-Mead`, puis on applique la méthodologie de l'annexe pour construire les profils de performance correspondants.

Information	**Valeur**
Newton-Raphson	beta = [-0.07000774 0.02381741], NLL = 12.26317655
Fisher	beta = [-0.07000774 0.02381741], NLL = 12.26317655
BFGS	beta = [-0.07000747 0.02381728], NLL = 12.26317655
Nelder-Mead	beta = [-0.07003106 0.02383049], NLL = 12.26317666



Les différentes méthodes d'optimisation conduisent à des estimations très proches des paramètres du modèle logistique. En particulier, Newton-Raphson, la méthode du score de Fisher et BFGS fournissent pratiquement la même valeur de la négative log-vraisemblance, ce qui indique qu'elles convergent vers la même solution. La méthode de Nelder-Mead aboutit également à une solution voisine, mais légèrement moins précise numériquement. Les probabilités ajustées restent comprises dans un intervalle modéré, ce qui montre que, pour ce jeu de données et dans la formulation retenue sans intercept, le modèle ne sépare pas parfaitement les deux groupes. Cela n'est pas incohérent : le modèle logistique fournit ici une tendance probabiliste globale plutôt qu'une classification tranchée. Les variables explicatives conservent donc un pouvoir explicatif limité mais réel.

Algorithme	Taux de succès	Moyenne	Médiane	Minimum	Maximum
Newton-Raphson	100.00%	0.001320s	0.001198s	0.000390s	0.003651s
Fisher	100.00%	0.001251s	0.001126s	0.000335s	0.003068s
BFGS	100.00%	0.005702s	0.004580s	0.002158s	0.018700s
Nelder-Mead	46.25%	0.002197s	0.002026s	0.001455s	0.003727s



Le profil de performance met en évidence une domination globale de la méthode du score de Fisher, dont la courbe est la plus élevée sur l'essentiel du domaine étudié. La méthode de Newton-Raphson présente également d'excellentes performances et reste très proche de Fisher, ce qui est cohérent avec la parenté théorique de ces deux procédures dans le cadre de la régression logistique.

La méthode BFGS apparaît moins compétitive, bien qu'elle conserve de bonnes propriétés de robustesse. En revanche, Nelder-Mead est nettement moins performant : sa courbe croît plus lentement et son taux de succès est plus faible, ce qui indique qu'il est à la fois moins efficace et moins robuste sur cette famille de problèmes.

Ainsi, pour cet exercice, les méthodes exploitant l'information dérivée (Newton-Raphson et Fisher) sont les plus adaptées. Elles résolvent le problème plus efficacement que les méthodes générales d'optimisation, en particulier que Nelder-Mead.

6.7 Conclusion générale

L'ensemble du projet montre qu'un même problème peut être abordé à plusieurs niveaux : **modélisation**, **conception algorithmique**, **implémentation numérique** et **évaluation expérimentale**.

Les expériences réalisées conduisent à plusieurs constats structurants :

- sur la fonction de Rosenbrock, les méthodes exploitant l'information dérivée, en particulier

`Newton-Raphson`, dominent globalement la comparaison finale, tandis que la méthode de la plus grande pente est pénalisée par la géométrie en vallée étroite ;

- pour les problèmes de vraisemblance univariée, `Newton-Raphson` est souvent très efficace, mais les méthodes de recherche directe restent pertinentes comme procédures de référence ou de validation ;
- dans les modèles à structure latente, comme les mélanges gaussiens, une méthode spécialisée comme `EM` bénéficie d'un avantage clair lorsqu'elle est comparée à des algorithmes d'optimisation plus généraux ;
- dans les exercices 11 et 12, les méthodes construites à partir des dérivées du problème (`Gauss-Newton`, `Levenberg-Marquardt`, `Newton-Raphson`, `Fisher`) apparaissent globalement les plus adaptées.

Au-delà des résultats propres à chaque exercice, le fil conducteur du projet est la **méthodologie de l'annexe**. En imposant une définition rigoureuse des quantités $\tau_{p,\alpha}$, T_p, $\rho_{p,\alpha}$ et $P_\alpha(\pi)$, elle fournit un cadre d'analyse qui permet de comparer les algorithmes selon deux critères complémentaires :

$$\text{efficacité} \quad \leftrightarrow \quad \pi \approx 1, \qquad \text{robustesse} \quad \leftrightarrow \quad \pi \approx R.$$

Le rapport ne se limite donc pas à juxtaposer des sorties numériques : il met en évidence ce que nous apprenons à faire en optimisation numérique, à savoir **formuler un problème, choisir une méthode adaptée, justifier la comparaison expérimentale et interpréter rigoureusement les résultats obtenus**.